

Paper Summary:
ReLU-QP: A GPU-Accelerated QP Solver for MPC (ICRA '24)

Da Ma

Overview

ReLU-QP is a GPU-accelerated Quadratic Programming (QP) solver designed for solving large-scale Model Predictive Control (MPC) problems efficiently in real time. It reformulates the Alternating Direction Method of Multipliers (ADMM) algorithm as a deep, weight-tied neural network with ReLU-style activation (box projection), enabling efficient execution using GPU-optimized matrix operations.

1. Original QP Formulation

We aim to solve the following constrained convex QP:

$$\begin{aligned} \min_y \quad & \frac{1}{2}y^\top Hy + g^\top y \\ \text{s.t.} \quad & c \leq Gy \leq d \end{aligned}$$

where:

- $H \in \mathbb{R}^{n \times n}$ is symmetric positive definite,
- $g \in \mathbb{R}^n$ is the cost gradient,
- $G \in \mathbb{R}^{m \times n}$ defines m linear constraints.

2. ADMM Reformulation

To apply ADMM, we introduce a splitting variable $z = Gy$ and form the augmented Lagrangian:

$$\mathcal{L}_\rho(y, z, \lambda) = \frac{1}{2}y^\top Hy + g^\top y + \lambda^\top (Gy - z) + \frac{\rho}{2}\|Gy - z\|^2$$

Each iteration of ADMM performs:

$$\begin{aligned} y^{(k+1)} &= \arg \min_y \left[\frac{1}{2}y^\top Hy + g^\top y + \frac{\rho}{2}\|Gy - z^{(k)} + \rho^{-1}\lambda^{(k)}\|^2 \right] \\ z^{(k+1)} &= \Pi_{[c,d]} \left(Gy^{(k+1)} + \rho^{-1}\lambda^{(k)} \right) \\ \lambda^{(k+1)} &= \lambda^{(k)} + \rho(Gy^{(k+1)} - z^{(k+1)}) \end{aligned}$$

This iterative process seeks to find a y that minimizes cost while satisfying the constraints by balancing between the original objective and a constraint penalty.

3. ReLU-QP Reformulation

ReLU-QP reorders the ADMM updates by updating λ before y . This is valid because ADMM allows flexibility in update order as long as consistency is preserved. While this introduces a one-step lag in λ , it does not affect convergence significantly. Reordering enables simplification into a single linear operation with a box projection:

$$v^{(k+1)} = \text{clamp}(Wv^{(k)} + b, \tilde{c}, \tilde{d})$$

with $v = \begin{bmatrix} y \\ z \\ \lambda \end{bmatrix}$. The clamp operator applies box projection only to the z components.

Matrix Definitions

Let $D = (H + \sigma I + G^\top \rho G)^{-1}$. Then:

$$W = \begin{bmatrix} D(\sigma I - G^\top \rho G) & 2DG^\top \rho & -DG^\top \\ GD(\sigma I - G^\top \rho G) + G & 2GDG^\top \rho - I & -GDG^\top + \rho^{-1} \\ \rho G & -\rho I & I \end{bmatrix}, \quad b = \begin{bmatrix} -Dg \\ -GDg \\ 0 \end{bmatrix}$$

4. Convergence and Stopping Criteria

Despite the reformulation, ReLU-QP uses the same residual-based stopping criteria as standard ADMM:

$$\begin{aligned} r^{\text{prim}} &= Gy - z, & (\text{primal residual}) \\ r^{\text{dual}} &= Hy + g + G^\top \lambda, & (\text{dual residual}) \\ \|r^{\text{prim}}\|_\infty &\leq \varepsilon_{\text{prim}}, & \|r^{\text{dual}}\|_\infty \leq \varepsilon_{\text{dual}} \end{aligned}$$

ReLU-QP checks these residuals periodically (e.g., every 25 iterations), since evaluating them is more expensive than the matrix step.

5. Precomputable Components

If the problem structure is fixed (e.g., in dense MPC), the following can be precomputed:

- $D = (H + \sigma I + G^\top \rho G)^{-1}$
- Weight matrix W and bias vector b
- Clamp bounds $\tilde{c}, \tilde{d}$, with box constraints on z only
- Preconditioned variants for a bank of ρ values

This allows for extremely efficient iteration: just compute $v^{(k+1)} = \text{clamp}(Wv^{(k)} + b)$ using GPU-friendly matrix ops.

6. Additional Insights

- Although $Gy \in [c, d]$ may hold, we must also satisfy $Gy = z$ for convergence.
- The ADMM minimization step optimizes a penalized objective, not the original cost, so iterative refinement is required.
- λ evolves to push y towards satisfying constraints exactly.
- The elegance of ReLU-QP is that each iteration is structurally equivalent to one layer of a deep network.

7. Summary of Benefits

- Entire ADMM logic embedded in a single matrix operation + projection
- Highly parallelizable on GPUs
- Compatible with neural network libraries like PyTorch
- Effective for solving large MPC problems at kHz rates
- Convergence properties inherited from ADMM